Quantum Implementation of MD5

Sangmin Cha[1][0009−0003−3614−6247], Gyeongju Song[2][0000−0002−4337−1843], Seyoung Yoon[3][0009−0008−3254−9256], and Hwajeong Seo[4][0000−0003−0069−9061]

Hansung University, South Korea
{xhxhdwkdpsyentific,thdrudwn98,sebbang99,hwajeong84}@gmail.com

Abstract. Quantum attacks such as Grover's algorithm reduce the security of classical hash functions such as MD5. In this paper, we present an efficient quantum circuit for the MD5 hash function and apply Grover's algorithm to perform an effective pre-image attack. Our MD5 quantum circuit first focuses on reducing quantum circuit depth, while also considering the number of qubits. To this end, we applied Draper's carry-lookahead adder to the MD5 quantum circuit and modified its structure. When Draper's adder was applied to the proposed MD5 structure, there was an approximately 81.4% reduction compared to before application. As a result, the proposed MD5 quantum circuit uses only 858 qubits and has a depth of 7,598.

Keywords: Quantum Computer · Grover's Algorithm · Quantum Collision Search · MD5 Hash Function · Post-quantum Security

1 Introduction

Quantum computers are computing devices that leverage quantum mechanical phenomena to perform computations fundamentally different from classical computers. First introduced by Richard Feynman in 1982[1], quantum computers have been shown to solve certain problems—such as integer factorization and discrete logarithm problems—exponentially faster than their classical counterparts (e.g., Shor's Algorithm) [2].

In cryptography, the advent of quantum computing has ushered in a new era known as Post-Quantum Cryptography (PQC). This has given rise to two major research directions: one focuses on mounting quantum attacks against existing cryptographic schemes, while the other aims to develop quantum-resistant algorithms that remain secure in the presence of quantum adversaries. Among the various quantum attacks, Grover's algorithm is known to be effective against symmetric key cryptosystems and hash functions [3]. In particular, Grover's algorithm enables a quantum search attack on an n-bit hash function with a time complexity of approximately $O(2^{n/2})$.

To apply Grover's algorithm to a hash function, the function must be expressed as a quantum circuit composed of quantum gates—the fundamental operations in quantum computation. The performance of such quantum circuits is typically evaluated based on two key metrics: the number of qubits used and

the circuit depth. Qubits are the quantum analogues of classical bits, while circuit depth refers to the number of sequential operations (or layers) required to complete the computation. Accordingly, quantum circuit optimization focuses on reducing either the number of qubits or the overall circuit depth.

In this study, we present a depth-optimized quantum circuit implementation of MD5 (i.e., Message Digest 5), a widely used classical hash function, for use with Grover's algorithm. We specifically focus on minimizing the circuit depth while maintaining reasonable qubit usage, and provide detailed performance evaluation of the resulting quantum implementation.

This paper is organized into five sections as follows. Section 2 reviews previous research and presents the foundational concepts necessary to understand this work. Section 3 describes the proposed implementation and the optimization techniques applied. Section 4 presents the performance evaluation of the implemented quantum circuit. Finally, Section 5 concludes the paper.

1.1 Contributions

The main contributions of this study are as follows:

1. **Depth-Optimized Quantum Implementation of MD5:** This paper presents the first depth-optimized quantum circuit of the MD5 hash function. In our design, we focus first on reducing quantum circuit depth, while also considering the number of qubits. As a result, the proposed MD5 quantum circuit uses only 858 qubits and achieves a depth of 7,598.
2. **Efficient Quantum Adder Selection and Application:** A predictive adder known as Draper's adder is adopted to significantly reduce the depth of the quantum addition circuit, which is a core operation in MD5 hash function. Compared to the usage of the qubit-optimized quantum adder, this choice reduces the depth by approximately 81.4%.

2 Related Works

Before discussing the quantum circuit implementation of MD5, it is necessary to discuss quantum computers, quantum gates, and the MD5 hash function. Therefore, this chapter will discuss the algorithm of MD5 in Section 2.1, an overview of quantum computers and the concept of qubit, representative quantum gates, and Grover's algorithm in Section 2.2, and finally a discussion of previously studied quantum circuit implementations of MD5 in Section 2.3.

2.1 MD5 Hash Function

MD5 is a cryptographic hash function invented by Ronald Rivest in 1992 and defined in RFC 1321[4]. Due to advances in computing power, its use as a security factor has been discouraged since around 2005, and as of 2007, it is already known that a collision pair can be found within seconds using a personal desktop/laptop computer[5].

MD5 has a 128-bit output and is based on the Merkle-Damgård structure, which divides the input message into blocks of 512 bits and combines it with the result of the previous block after 64 rounds of computation for each 512-bit block.

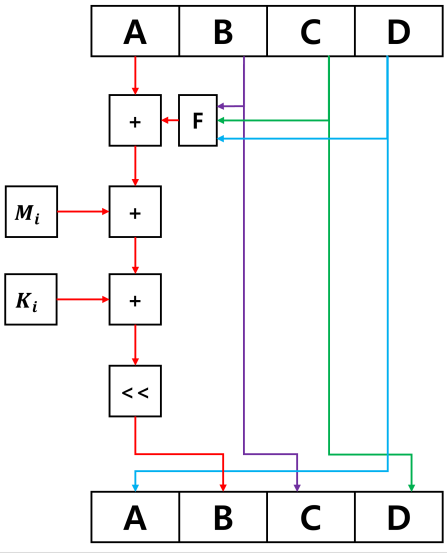

Fig. 1. One-round data flow in the MD5 compression function.

The 64 rounds of computation can be represented as shown in Figure 1. It consists of a combination of 32-bit integer modulo addition operations, a nonlinear function `F`, and a left cyclic shift operation. The nonlinear function `F` is further divided into four functions called `F`, `G`, `H`, and `I`. The number of rounds determines the use of one of the four functions. The exact content of each function is shown in Algorithm 1.

Finally, MD5 combines four 32-bit internal state variables into a little-endian to produce 128 bits of hash output, which the `LE32()` function in Algorithm 2 shows converting to a little-endian. The complete structure of the MD5 algorithm, including the rounds above, can be represented as a pseudocode as shown in Algorithm 2.

2.2 Quantum Circuits and Fundamental Gates

As introduced earlier, quantum computing, a field introduced around 1982, can be defined as a computer that uses quantum mechanical phenomena to perform computations and is currently being researched, developed, and invested in by various countries and companies for practical use and commercialization. A qubit

Algorithm 1 Internal Functions of MD5

```
1: procedure FuncF(B, C, D)
2:     return (B ∧ C) ∨ (¬B ∧ D)
3: end procedure
4:
5: procedure FuncG(B, C, D)
6:     return (B ∧ D) ∨ (C ∧ ¬D)
7: end procedure
8: procedure FuncH(B, C, D)
9:     return (B ⊗ C ⊗ D)
10: end procedure
11:
12: procedure FuncI(B, C, D)
13:     return (C ⊗ (B ∨ ¬D))
14: end procedure
```

is the smallest unit of information in a quantum computer, corresponding to a bit in a traditional computer. Bra-Kets Notation is used as a way to represent qubits, and representative qubits are $|0\rangle$ and $|1\rangle$. A quantum gate can be defined as a fundamental operation in a quantum computer. Quantum gates are sometimes represented as matrices. Hadamard gates, or H-gates, are responsible for putting one qubit into superposition, a quantum mechanical phenomenon, and are commonly used in quantum circuits to generate random values. The Pauli-X gate or X gate inverts the state of one qubit and corresponds to the bit-NOT operation in a typical computer. The Controlled-X gate or CNOT gate takes two qubits as input, a control qubit and a target qubit, and entangles the state of the control qubit with the state of the target qubit, so that if the state of the control qubit is $|1\rangle$, the X gate operation is performed on the target qubit. The Toffoli gate, or CCNOT gate, takes two control qubits and one target qubit as inputs and performs an X gate operation on the target qubit when both input qubits have a state of $|1\rangle$. As with Toffoli gates, there can be multiple control qubits, and they are often named C (Controlled) for the number of control qubits, such as CCC...NOT gates. The Grover algorithm is a quantum algorithm that can be used for search problems and is defined as an algorithm that finds, with high probability, a unique input that produces a specific output value in an unstructured search. There are no special restrictions on the black-box functions that Grover's algorithm can target, and it is known to be able to perform exploration in $O(\sqrt{N})$ time complexity for problems that have $O(N)$ time complexity with classical algorithms. These properties allow Grover's algorithm to be used to attack the pre-image resistance of hash functions.

2.3 Previous Quantum Implementations of MD5

Prodipto et al.[6] implemented MD5 and SHA1 as quantum circuits, and the basic operations used in MD5, such as NOT, OR, AND, and XOR, were implemented as quantum circuits and applied to the general structure of MD5, and the adders for carry and sum were used by implementing a truth table-based implementation. In addition, the remaining operations for 2^n were optimized as an AND operation with (2^n)-1, and the overall optimization was aimed at minimizing the use of qubits. IBM's Qiskit development environment was used.

Algorithm 2 MD5 Hash Function

procedure MD5(*msg*)
#**Padd** :
padded ← *msg*
Append bit `"1"` to *padded*
while bitlen(*padded*) mod 512 ≠ 448 **do**
  Append bit `"0"` to *padded*
end while
Append int64(bitlen(*msg*)) (little-endian) to *padded*

#**Initialize** $k[64]$:
for $i \leftarrow 0$ to 63 **do**
  $k[i] \leftarrow \lfloor |\sin(i+1)| \cdot 2^{32} \rfloor$
end for

#**Initialize** $s[64]$:
$s[64] \leftarrow [7, 12, 17, 22, \dots]$

#**Initialize** a_0, b_0, c_0, d_0 :
$a_0 \leftarrow$ `0x67452301`
$b_0 \leftarrow$ `0xefcdab89`
$c_0 \leftarrow$ `0x98badcfe`
$d_0 \leftarrow$ `0x10325476`

#**Main loop**
for $i \leftarrow 0$ to bitlen(*padded*) step 512 **do**
  $M[0..15] \leftarrow$ Split padded block into 16 32-bit little-endian words
  $A \leftarrow a_0$, $B \leftarrow b_0$, $C \leftarrow c_0$, $D \leftarrow d_0$
  #**64 Rounds**
  for $j \leftarrow 0$ to 63 **do**
    if $0 \le j \le 15$ **then**
      $F \leftarrow \text{FuncF}(B, C, D)$
      $g \leftarrow j$
    else if $16 \le j \le 31$ **then**
      $F \leftarrow \text{FuncG}(B, C, D)$
      $g \leftarrow (5j + 1) \bmod 16$
    else if $32 \le j \le 47$ **then**
      $F \leftarrow \text{FuncH}(B, C, D)$
      $g \leftarrow (3j + 5) \bmod 16$
    else
      $F \leftarrow \text{FuncI}(B, C, D)$
      $g \leftarrow (7j) \bmod 16$
    end if
    $temp \leftarrow D$
    $D \leftarrow C$
    $C \leftarrow B$
    $B \leftarrow (B + \text{LeftCircularShift}(A + F + k[j] + M[g], s[j])) \bmod 2^{32}$
    $A \leftarrow temp$
  end for
  $a_0 \leftarrow (a_0 + A) \bmod 2^{32}$
  $b_0 \leftarrow (b_0 + B) \bmod 2^{32}$
  $c_0 \leftarrow (c_0 + C) \bmod 2^{32}$
  $d_0 \leftarrow (d_0 + D) \bmod 2^{32}$
end for
  return LE32(a_0) ‖ LE32(b_0) ‖ LE32(c_0) ‖ LE32(d_0)
end procedure

R. Preston[7] implemented MD5, SHA-1, SHA-2, and SHA-3 as quantum oracles to drive Grover's algorithm and calculated the resources required to attack with Grover's algorithm. In addition, the implementation of quantum MD5 was designed to minimize the use of qubits. Specifically, they used an adder that does not use any auxiliary qubits [8] for the adder, and the nonlinear function F of MD5 was implemented as an in-place operation and implemented using Microsoft's Q# development environment.

3 Proposed Method

This section discusses the development environment used for implementation, the implementation of basic operations such as logical operators and addition operations, and the complete implementation of the MD5 algorithm using it.

3.1 Quantum Implementations of Basic Logical and Assignment Gates

Algorithm 3 Quantum Logical Operators

```
1: procedure OperatorNot(q1)
2: X | q1;
3: end procedure
4:
5: procedure OperatorAnd(qOut, q1, q2)
6: Toffoli | (q1, q2, qOut)
7: end procedure
8:
9: procedure OperatorOr(qOut, q1, q2)
10: X | q1;
11: X | q2;
12: Toffoli | (q1, q2, qOut)
13: X | q1;
14: X | q2;
15: X | qOut;
16: end procedure
17:
18: procedure OperatorXor(qOut, q1, q2)
19: CNOT | (q1, qOut)
20: CNOT | (q2, qOut)
21: end procedure
```

The algorithm 3 illustrates the implementation of the logical operators NOT, AND, OR, and XOR as quantum circuits. A bitwise NOT operation is performed using a single X gate applied to the target qubit, corresponding to `OperatorNot()`. The bitwise AND operation is implemented using a single Toffoli gate with two input qubits as control qubits, corresponding to `OperatorAnd()`. The bitwise OR operation is implemented by `OperatorOr()`, and the bitwise XOR operations are implemented using two CNOT gates, corresponding to `OperatorXor()`.

Operations that assign values to qubits can be categorized into those that assign classic values and those that assign values from other qubits. Operations that assign a classic value can "assign" a value to a qubit variable by repeating the process of performing an X-gate operation on the corresponding number of digits of the qubit when each bit of the classic value has a value of 1. This corresponds to the `AssignmentQI()` operation in lines 1 through 8 of Algorithm 4. The operation of assigning a value from another qubit can be done by performing CNOT gate operations on the qubits corresponding to each digit. This corresponds to the `Assignment2Q()` operation in lines 10 through 14 of Algorithm 4. Swapping the values of two qubits can be done as an iteration of the Swap gate, which corresponds to the `ValueSwap2Q()` operation in lines 16 through 20 of Algorithm 4. There are various methods for implementing left-circular shift operations, but in this study, we implemented it based on formula 1 which is a well-known method in classic computers. Note that w is a number of total bits, e.g. 32 or 64.

$$ROT(n, k) = (n << k)|(n >> (w - k)) \tag{1}$$

The implementation of formula 1 corresponds to the `LeftCircularShift()` operation in lines 22 to 33 of algorithm 4. It should be noted that equation 1

Algorithm 4 Quantum Operators for Quantum Implementation of MD5

procedure AssignmentQI(*op1, op2*)
for $i \leftarrow 0$ to 31 **do**
  $b \leftarrow (op2\rangle\rangle i) \wedge 1$
  if $b = 1$ **then**
    X | op1[i]
  end if
end for
end procedure

procedure Assignment2Q(*op1, op2*)
for $i \leftarrow 0$ to 31 **do**
  CNOT | (op2[i] ,op1[i])
end for
end procedure

procedure ValueSwap2Q(*op1, op2*)
for $i \leftarrow 0$ to 31 **do**
  Swap | (op1[i] ,op2[i])
end for
end procedure

procedure LeftCircularShift(*op1, amount*)
$d \leftarrow amount\%32$
for $i \leftarrow 0$ to $(d//2) - 1$ **do**
  Swap | $(op1[32 - d + i], op1[31 - i])$
end for
for $i \leftarrow 0$ to $((32 - d)//2) - 1$ **do**
  Swap | $(op1[i], op1[31 - d - i])$
end for
for $i \leftarrow 0$ to 15 **do**
  Swap | $(op1[i], op1[31 - i])$
end for
end procedure

is based on the general notation in which the bit indexing of the target number n is expressed with the most significant bit index as 0. However, in the ProjectQ Framework used in this study, the least significant bit index is expressed as 0. Therefore, it is important to note that the "left" in left shift is defined in the opposite way. This means that the `LeftCircularShift()` in Algorithm 4 is also implemented with this consideration. The implementation of `LeftCircularShift()` can be divided into three parts. The first for loop corresponds to (n >> (w - k)) in formula 1 and internally inverts the bit block that will be moved to the front after the circular shift, ensuring the correct relative order during the subsequent inversion step. The second for loop corresponds to (n << k) and internally inverts the bit block that will be pushed to the back after the shift, ensuring that the correct order is maintained after the overall inversion. The final for loop completes the left circular shift operation by reversing the entire array, thereby exchanging the relative positions of the two bit blocks, after the two terms of the formula have been properly internally aligned through the previous two for loops. For the addition operations, we used Draper's quantum adder [9] to minimize the circuit depth. Draper's adder requires a total of 58 additional bits (32 carry qubits and 26 ancilla qubits) to perform a 32-bit operation, but it can process a single 32-bit addition operation with a depth of 29. On the other hand, a method that reuses qubits to perform additions without additional qubits [8] is relatively inefficient, requiring a depth of 156. When performing the entire operation based on the MD5 algorithm, the Draper adder results in a depth of $29 \times (64 \times 4 + 4) = 29 \times 260 = 7540$ from the adder. On the other hand, the method in [8] results in a depth from the adder of $156 \times (64 \times 4 + 4) = 156 \times 260 = 40560$. From this, we can see that the depth

when using the Draper adder is about 18.6% of the depth of the method in [8], which is a reduction of about 81.4% due to the choice of adder alone. The two addition functions `AddQ()` and `AddK()` that appear in the subsequent pseudocode are Draper's adders, The difference between the two functions is whether the numbers to be added are input in the form of qubits or classic values. Except for the part where the control gate, if statement, and X gate are used to implement the parts involving the operand qubits/bits, they have the same structure and code.

3.2 Quantum Implementation of MD5

This paper first focuses on reducing circuit depth, and then considers minimizing the number of qubits required in the quantum circuit. Algorithm 5 presents the pseudocode of the MD5 quantum circuit.

Among the qubit registers appearing in Algorithm 5, `a0`, `b0`, `c0`, `d0`, `A`, `B`, `C`, `D`, `temp1`, and `temp2` are 32 qubits, `ancilla` is 26 qubits, and `chunk512` is 512 qubits. Among the functions used in the pseudocode Algorithm 5, the `Compute()`, and `Uncompute()` functions are supported by the ProjectQ framework, and their roles are as follows. The `Compute(backend)` and `Uncompute(backend)` functions work in pairs, where the quantum operations in the block declared with `Compute()` are inversed when `Uncompute()` is executed, resulting in the value of the used qubits being returned to the value before `Compute()` is executed. The `LE32()` function does the same thing as shown in Algorithm 2. The `Reversed()` function is supported as Python's embedded function and used to reverse the input list type object.

The `FuncF()`, `FuncG()`, `FuncH()`, and `FuncI()` functions in Lines 18-30 of Algorithm 5 are quantum circuit implementations of the nonlinear functions of MD5. The F function uses 32 bits of temp qubits to perform all 32-bit operations on x, y, and z in parallel. At this point, `temp1` is cleaned to 0. In the addition operation stage immediately following the F function, `temp1` used in F is reused as the carry in Draper's adder to reduce the total qubit usage.

From line 10 to line 41 means MD5 operations for each input message chunk per 512 bits. Line 10 to line 13 do preprocessing and assignment as qubit for the 512 bit message chunk. This allows us to use with Grover's algorithm because the algorithm uses Hadamard gates to superpositioning for target algorithm's input at the start so it is necessary to put the input message to quantum state. Line 32 determines a specific 32 bit message from total 512 bit chunk based on the round number and renames as `M`. Line 33 do addition with the result of nonlinear function to state A and following `Uncompute()` do its reverse operation so the qubit `temp1` is back to zero qubits for later use. Line 35 do addition with the message `M` to state `A`. Line 36 do addition with pre-defined constant `K`, which is classic value. From line 38 to 41 do swap states as last operation per each MD5 round. The one round of MD5 implementation can be described as in Figure 2.

Algorithm 5 MD5 Hash Function in Quantum Circuits

procedure MD5(msg)
#same as classic MD5 before initializing (a_0, b_0, c_0, d_0)
#**Initialize** a_0, b_0, c_0, d_0 :
$a_0 \leftarrow$ `0x67452301`
$b_0 \leftarrow$ `0xefcdab89`
$c_0 \leftarrow$ `0x98badcfe`
$d_0 \leftarrow$ `0x10325476`
#**Main loop**
for $i \leftarrow 0$ to bitlen($padded$) step 512 **do**
  $chunk512 \leftarrow$ padded
  for $j \leftarrow 0$ to 15 **do**
    $chunk512[j * 32 : (j + 1) * 32] \leftarrow$ LE32(chunk512[j*32:(j+1)*32])
  end for
  $A \leftarrow a_0$, $B \leftarrow b_0$, $C \leftarrow c_0$, $D \leftarrow d_0$
  #**64 Rounds**
  for $j \leftarrow 0$ to 63 **do**
    Compute(backend){
    if $0 \leq j \leq 15$ **then**
      $temp2 \leftarrow \text{FuncF}(B, C, D, temp1)$
      $g \leftarrow j$
    else if $16 \leq j \leq 31$ **then**
      $temp2 \leftarrow \text{FuncG}(B, C, D, temp1)$
      $g \leftarrow (5j + 1) \bmod 16$
    else if $32 \leq j \leq 47$ **then**
      $temp2 \leftarrow \text{FuncH}(B, C, D, temp1)$
      $g \leftarrow (3j + 5) \bmod 16$
    else
      $temp2 \leftarrow \text{FuncI}(B, C, D, temp1)$
      $g \leftarrow (7j) \bmod 16$
    end if
    }
    $M \leftarrow$ Reversed(chunk512[(g*32):(g+1)*32])
    $A \leftarrow \text{AddQ}(A, temp2, temp1, ancilla)$
    Uncompute($backend$)
    $A \leftarrow \text{AddQ}(A, M, temp1, ancilla)$
    $A \leftarrow \text{AddC}(A, k[j], temp1, ancilla)$
    $A \leftarrow A <<< s[j]$
    $A \leftarrow \text{AddQ}(A, B, temp1, ancilla)$
    Swap(A, D)
    Swap(C, D)
    Swap(B, C)
  end for
  $a_0 \leftarrow \text{AddQ}(a_0, A, temp1, ancilla)$
  $b_0 \leftarrow \text{AddQ}(b_0, B, temp1, ancilla)$
  $c_0 \leftarrow \text{AddQ}(c_0, C, temp1, ancilla)$
  $d_0 \leftarrow \text{AddQ}(d_0, D, temp1, ancilla)$
end for
return LE32(a_0) $\|$ LE32(b_0) $\|$ LE32(c_0) $\|$ LE32(d_0)
end procedure

Algorithm 6 Internal Functions of MD5 in Quantum Circuits

```
 1: procedure FuncF(result, x, y, z, temp)
 2: for i ← 0 to 31 do
 3:     CNOT | (y[i], temp[i])
 4:     CNOT | (z[i], temp[i])
 5:     Toffoli | (x[i], temp[i], result[i])
 6:     CNOT | (y[i], temp[i])
 7:     CNOT | (z[i], temp[i])
 8:     CNOT | (z[i], result[i])
 9: end for
10: end procedure
11:
12: procedure FuncG(result, x, y, z, temp)
13: for i ← 0 to 31 do
14:     CNOT | (x[i], temp[i])
15:     CNOT | (y[i], temp[i])
16:     Toffoli | (z[i], temp[i], result[i])
17:     CNOT | (x[i], temp[i])
18:     CNOT | (y[i], temp[i])
19:     CNOT | (y[i], result[i])
20: end for
21: end procedure
22:
23: procedure FuncH(result, x, y, z)
24: for i ← 0 to 31 do
25:     CNOT | (x[i], result[i])
26:     CNOT | (y[i], result[i])
27:     CNOT | (z[i], result[i])
28: end for
29: end procedure
30:
31: procedure FuncI(result, x, y, z)
32: for i ← 0 to 31 do
33:     CNOT | (y[i], result[i])
34:     X | x[i]
35:     Toffoli | (x[i], z[i], result[i])
36:     X | x[i]
37:     X | result[i]
38: end for
39: end procedure
```

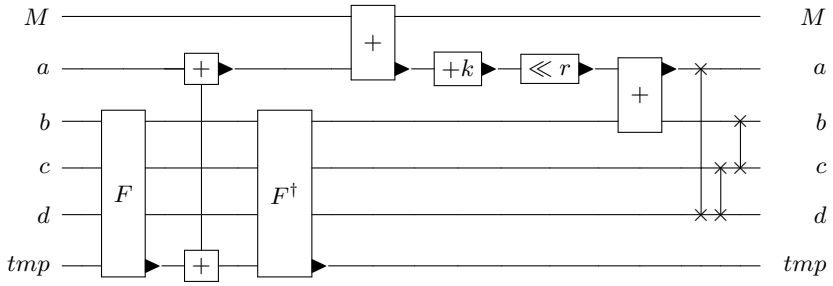

Fig. 2. Quantum implementation of the MD5 round function.

4 Evaluation

In this Section, we analyze the quantum resource usage of our quantum MD5 implementation and highlight its differences from previous studies on quantum MD5 circuits.

Quantum circuits can be implemented using various programming languages and quantum computing frameworks. In this study, we used the Python programming language in conjunction with the ProjectQ framework[10]. A classical simulator was used as the backend to execute the quantum circuit.

4.1 Quantum Resource Analysis of MD5

The quantum resource usage of the implemented quantum MD5 is shown in Table 1.

Table 1. Quantum resources for MD5 hash functions.

Algorithm	Qubit	Quantum gates			
		Toffoli	CNOT	X	Depth
MD5	858	68,784	41,420	19,090	7,598

The following analysis is possible from the qubit perspective. The total number of qubits used, 858, can be broken down into 512 + 26 + 32 * 10, and each qubit count is necessary for the following purposes. The 512 qubits represent the unit size of 512 bits used by MD5 to divide and process the entire input message of variable length into multiple parts. These qubits store the potential original message to be calculated using Grover's algorithm. Unlike later hash functions such as SHA-1, MD5 does not have a message expansion function that reconstructs the initial message through a series of rules, so it is not easy to save additional qubits in this part. Although there is a rule that the last 512-bit block must contain at least 65 bits of padding information, this also depends on the length of the input message, so it could not be utilized. The following 26 qubits are the number of auxiliary qubits required by Draper's adder. This depends on the size of the adder's input, but since only 32-bit addition is used in MD5, 26 qubits are required. Finally, the last 10 32-qubit spaces can be divided into 8 32-bit state variables and 2 temporary variables. The eight state variables are the 8 states defined in MD5: `a0`, `b0`, `c0`, `d0`, and `a`, `b`, `c`, `d`, while the two temporary variables are used for temporary purposes such as storing the carry in the adder and reducing the circuit depth in the nonlinear function `F`.

The following analysis is possible in terms of depth. The total depth of 7598 can be expressed as 7213 + 385. First, 7213 is the depth measured in the adder section, which is smaller than 7540, the product of 29 (the depth of a single 32-bit addition operation) and 260 (the total number of addition operations). However, this appears to be the result of the optimization performed by the ProjectQ framework. The remaining 385 corresponds to the depth of all other components excluding the adder, and this is also smaller than (depth of each individual component * number of executions) for the same reason. The reason why the adder accounts for most of the depth and the remaining components account for a much smaller proportion of the depth is that the adder requires the order of operations between individual qubits in a 32-qubit input, but other components, such as the nonlinear function F, have independent operations between individual qubits, resulting in a much lower depth.

A comparison of the quantum resource usage of other hash functions is shown in Table 2.

Table 2. Comparison of quantum resource requirements for various hash functions.

Hash Function	MD5	SHA-3	PHOTON	ASCON	Xoodyak	ESCH256	ESCH384
Qubit	858	3,200	18,944	35,136	14,464	769	1,025
Depth	7,598	10,128	3,371	2,487	760	95,033	182,421
X gate	19,090	85	10,369	97,346	27,754	10,559	20,248
CX gate	41,420	33,269,760	315,328	159,232	50,944	113,360	217,792
CCX gate	68,784	84,480	18,432	55,296	13,824	37,200	71,424

Of the hash functions in Table 2, PHOTON[11], ASCON[12], Xoodyak[13], ESCH256 and ESCH384[14] are lightweight hash functions. In addition, all hash functions except MD5 were measured using 256 bits of input. And all the content in Table 2 except MD5 is cited from [15].

Table 3 shows estimates of the quantum resources required when used in conjunction with Grover's algorithm.

Table 3. Quantum resource estimation of Grover's algorithm for MD5 hash function.

Algorithm	Quantum gates			
	Toffoli	CNOT	X	Depth
MD5	1.64×2^{79}	1.98×2^{78}	1.83×2^{77}	1.45×2^{76}

The figures in Table 3 were calculated by multiplying the usage of quantum resources for a single MD5 operation discussed earlier, that is, the contents of Table 1, by $((\pi/4) \times (2^{128/2}))$, which is the number of attacks by the Grover algorithm on MD5.

4.2 Comparison with Previous Works

Table 4 summarizes the differences between existing studies [6,7] on quantum implementations of MD5 and this study.

The information regarding qubit usage and circuit depth listed in Table 4 is provided for reference purposes only. Furthermore, the qubit usage and depth were compared based on the quantum resource usage when running the MD5 algorithm itself, not when using Grover's algorithm. Specifically, Prodipto Das et al.'s paper [6] did not mention depth, and regarding qubits, it stated that 32 qubits were used. Considering the overall structure of MD5 and the design required for the Grover algorithm, this appears to be a figure for a single round. Richard Preston's implementation [7] appears to have used 801 qubits, which seems a possible implementation result considering MD5's overall structure and the Grover algorithm. Regarding depth, the paper lists 120,000, a figure appearing in its graph. Given that this implementation is a qubit optimization implementation, this seems a possible result. However, it should be noted that since the development environments used in each implementation differ, the estimation methods for resource usage, including qubits and depth, may also vary.

Compared to existing implementations of MD5 [6,7], the key difference is that the implementation in this study is a depth-optimized implementation. It is known that there is a trade-off between qubit usage and circuit depth. In the implementation of MD5, the choice of the quantum adder plays a major role. Specifically, there are two implementation directions for quantum adders: minimizing the use of qubits by not using any qubits other than the operand qubits, or using a few more qubits but reducing the depth of the circuit. Since the main operation in MD5 is the adder, it can be said that the selected adder determines whether the optimization direction of quantum MD5 is qubit-oriented or depth-oriented.

Table 4. Comparison with previous quantum implementations of MD5.

Implementation	Ours	Prodipto Das et al. [6]	Richard Preston [7]
Optimization	depth	qubit	qubit
Adder	carry-lookahead [9]	truth table based ripple-carry	ripple-carry [8]
Qubits	858	32 (one round)	801
Depth	7,598	unknown	120,000

5 Conclusion

The development of quantum computers poses a major threat to existing cryptography that does not take into account quantum attacks, including MD5. While it is well known that MD5 is vulnerable to quantum attacks such as Grover's algorithm, with its theoretical time complexity drastically reduced, the exact level of threat requires an understanding of the quantum resources required for the attack.

Therefore, in this study, we comprehensively considered various levels, from individual computational components to the overall algorithm structure, and implemented MD5 as an optimized quantum circuit to provide step-by-step information on quantum resource usage. The structure we propose places the greatest emphasis on reducing the depth of the circuit, as this is a major constraint in current and near-future quantum hardware. By providing a depth-optimized MD5 quantum circuit implementation, this study contributes to both the perspective of cryptanalysis, which accelerates the potential timeline for effectively attacking the MD5 hash function using Grover's algorithm, and the design of quantum algorithms.

Furthermore, unlike previous studies that focused on reducing the number of qubits, this study concentrated on reducing the depth of the circuit and applied various optimization techniques, such as the selection of adders, shift operations, and the careful use of temporary qubits considering the overall structure. These techniques can be generalized and applied to the quantum implementation of other algorithms.

Our follow-up research will continue to study advanced MD5 quantum circuits that use fewer depth-qubits, as well as implement and analyze quantum circuits for hash functions that are currently actively used in various fields. Ultimately, we believe this research will contribute to the broader goal of developing quantum-friendly cryptanalysis techniques and pave the way for a more sophisticated understanding of resource balancing in quantum computing.

References

1. R. P. Feynman, "Simulating physics with computers," *International Journal of Theoretical Physics*, vol. 21, pp. 467–488, Jun 1982.
2. P. W. Shor, "Algorithms for quantum computation: discrete logarithms and factoring," in *Proceedings 35th annual symposium on foundations of computer science*, pp. 124–134, Ieee, 1994.
3. L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proceedings of the twenty-eighth annual ACM symposium on Theory of computing*, pp. 212–219, 1996.
4. R. L. Rivest, "The MD5 Message-Digest Algorithm." RFC 1321, Apr. 1992.
5. M. Stevens, "On collisions for MD5," *International Journal of Human-computer Studies / International Journal of Man-machine Studies - IJMMS*, 01 2007.
6. P. Das, S. Biswas, and S. Kanoo, "Quantum implementation of SHA1 and MD5 and comparison with classical algorithms," *Quantum Information Processing*, vol. 23, p. 176, May 2024.
7. R. H. Preston, "Applying Grover's algorithm to hash functions: A software perspective," *IEEE Transactions on Quantum Engineering*, vol. 3, p. 1–10, 2022.
8. Y. Takahashi, S. Tani, and N. Kunihiro, "Quantum addition circuits and unbounded fan-out," 2009.
9. T. G. Draper, S. A. Kutin, E. M. Rains, and K. M. Svore, "A logarithmic-depth quantum carry-lookahead adder," 2004.
10. D. S. Steiger, T. Häner, and M. Troyer, "ProjectQ: an open source software framework for quantum computing," *Quantum*, vol. 2, p. 49, Jan. 2018.
11. Z. Bao, A. Chakraborti, N. Datta, J. Guo, M. Nandi, T. Peyrin, and K. Yasuda, "Photon-beetle authenticated encryption and hash family," 2019.
12. C. Dobraunig, M. Eichlseder, F. Mendel, and M. Schläffer, "ASCON v1.2: Lightweight authenticated encryption and hashing," *J. Cryptol.*, vol. 34, July 2021.
13. J. Daemen, S. Hoffert, M. Peeters, G. V. Assche, and R. V. Keer, "Xoodyak, a lightweight cryptographic scheme," *IACR Transactions on Symmetric Cryptology*, vol. 2020, Special Issue 1, pp. 60–87, 2020.
14. C. Beierle, A. Biryukov, L. Cardoso dos Santos, J. Großschädl, L. Perrin, A. Udovenko, V. Velichkov, and Q. Wang, "Lightweight aead and hashing using the sparkle permutation family," *IACR Transactions on Symmetric Cryptology*, vol. 2020, pp. 208–261, June 2020.
15. W.-K. Lee, K. Jang, G. Song, H. Kim, S. O. Hwang, and H. Seo, "Efficient implementation of lightweight hash functions on gpu and quantum computers for IoT applications," *IEEE Access*, vol. 10, pp. 59661–59674, 2022.